Computer Programming 1

Laboratory

(command line with files)

(27-10-2026)

Exercise 2a: Argc and Argv

Implement multiply, a program that takes a sequence of numbers as command-line arguments and outputs their product

Exercise 2b: Argc and Argv

Implement *calculate*, a program that takes, as argument from the command line, one option that specifies the mathematical operation to do and then a sequence of numbers (max. 10 numbers) and performs the operation on the numbers

E.g.,

calculate --max 10 30 40 5

Note: calculate is the name of the executable file (the one that by default is a.out)

*In these slides the meaning of «as first argument» is: *argument in position 1 (not in position 0)*

Exercise 2c: Argc and Argv

Implement *calculate*, a program that takes, as argument from the command line, one option that specifies the mathematical operation to do and then a sequence of numbers (max. 10 numbers) and performs the operation on the numbers

E.g.,

calculate --max 10 30 40 5

Variant 1: extend the implementation by implementing and controlling the execution of different functions: --sum, --product, --max, --average

calculate --max 10 30 40 5

calculate --sum 10 30 40 5

Constraint: please, add checks to avoid crashes of the program in case of wrong inputs are interested by the user or passed by using the command line (e.g., a char instead of a number)

E.g., calculate --max 10 30 c 5

Exercise 2d: Argc and Argv

Implement *calculate*, a program that takes, as argument from the command line, one option that specifies the mathematical operation to do and then a sequence of numbers (max. 10 numbers) and performs the operation on the numbers

E.g.,

calculate --max 10 30 40 5

Variant 2: re-implement calculate, so that it takes as command line arguments the option, but if there are no numbers in the command lines, the program can now read the number from *cin* (one per line or separated by spaces, if you prefer).

E.g.,

calculate --max 10 30 c 5

calculate --max

Exercise 3a: RPN Calculator (1/3)

Implement a program, rpn, which takes as arguments an expression in RPN (Reverse polish notation) and outputs its result.

RPN: operands first and then operators.

For instance: *2 3 +*

Examples

- rpn 2 3 + 5 +
  - result → 10
- rpn 10 10 10 + +
  - result → 30
- rpn 2 3 + 5 *
  - result → 25
- rpn 10 + 10
  - result → ERROR

Suggestions:
- You can use an array as base structure
- You can fix a max size

Suggestions:
- Implement the advanced version based on *stack* (see the next slide the Exercise 3b)

Exercise 3b: RPN Calculator with Stack (2/3)

Given that a ***stack*** is a data structure which allows to access data in a LIFO fashion (Last-In First-Out)

A stack can be used using two functions:

- *push(number),* which pushes a number (i.e., element) in the stack
- *pop(),* which pops a number from the stack (or return an error if the stack is empty)Push the result in the stack

Step 1: Implement a program that presents to the user a menu of the possible functions that can be performed in the stack, and a loop to control the continuous execution of such functions on the stack

Example of an execution:

push(3)
push(4)
pop()
4
pop()
3

Exercise 3c: RPN Calculator with Stack (3/3)

Given that a ***stack*** is a data structure which allows to access data in a LIFO fashion (Last-In First-Out)

A stack can be used using two functions:

- *push(number),* which pushes a number (i.e., element) in the stack
- *pop(),* which pops a number from the stack (or return an error if the stack is empty)Push the result in the stack

Step2: Improve the RPN implementation by using a stack and the following strategy:

- When you find a number, push the number in the stack
- When you find an operator (e.g., +):
  - Pop the appropriate number of operands from the stack
  - Perform the operation
  - Push the result in the stack

Exercise 4: CUT *(1/2)*

Implement a program that simulates the behavior of the Linux **CUT** utility.

The program must read text from standard input line by line, split each line into fields (substrings) using a specified delimiter character, and print a selected field to standard output

The program must accept the following optional command-line arguments (in any order):

- -d<character>: Specifies the character to use as the field delimiter. Default: comma ","
- -f<number>: Specifies the index of the field to extract. Default: 1.
- --debug: Enables debug mode. When active, the program must print verbose information regarding parsed command-line arguments.

Constraint: implement a supporting function with the following signature:

void split(std::string fields[], std::string line, char delimiter);

The function must perform the following tasks:

- Reset or clear the array of *fields* (assume a maximum capacity of 100 fields per line).
- Iterate through line by searching for delimiter occurrences using the function *find*.
- Extract each field using *substr* and store it sequentially inside the fields array.
- Properly handle and extract the remaining substring following the last delimiter.

Exercise 4: CUT *(2/2)*

Implement a program that simulates the behavior of the Linux **CUT** utility.

The program must read text from standard input line by line, split each line into fields (substrings) using a specified delimiter character, and print a selected field to standard output

The program must accept the following optional command-line arguments (in any order):

- -d<character>: Specifies the character to use as the field delimiter. Default: comma “,”
- -f<number>: Specifies the index of the field to extract. Default: 1.
- --debug: Enables debug mode. When active, the program must print verbose information regarding parsed command-line arguments.

Example of executions:

input from the user->

```
$ ./a.out
mario,rossi,roma
mario
```

```
$ ./a.out -d: -f2
daemon:x:1:1
 x
```

```
$ ./a.out -d: -f3 --debug
daemon:x:1:1
Arguments:
g
3
LINE IS:
daemon:x:1:1daemon:x:1:1
FIELDS ARE:
daemon:x:1:1daemon:x:1:1
END FIELDS
```